

École Nationale Supérieure des Mines de Saint Étienne

Efficient heuristics and metaheuristics for the unrelated parallel machine scheduling problem with release dates and setup times

GP - Computer Science - Software Design

Projet Java

Élèves :
Salim KISMA Samy Taleb

Enseignant :
Pierre UNY

I Compréhension et synthèse

1 Article étudié

Efficient heuristics and metaheuristics for the unrelated parallel machine scheduling problem with release dates and setup times.

2 Présentation du problème

L'article traite d'un problème d'ordonnancement sur machines parallèles non reliées (*Unrelated Parallel Machine Scheduling Problem*, ou UPMSP) avec des dates de disponibilité et des temps de préparation dépendants de la séquence. Dans la notation de Graham, il est formulé comme :

$$R \mid r_j,\ s_{ijk} \mid C_{\max}$$

L'objectif est de minimiser la durée totale d'exécution appelée makespan ($C_{\max}$), c'est-à-dire le moment où la dernière tâche se termine.

On dispose de n tâches indépendantes à exécuter sur m machines parallèles non identiques $(M_1, M_2, \ldots, M_m)$. Chaque tâche j possède :

— une date de disponibilité r_j avant laquelle elle ne peut commencer,
— une durée d'exécution p_{jk} qui dépend de la machine k,
— un temps de setup s_{ijk} lorsqu'elle est exécutée après la tâche i sur la machine k.

Le but est de déterminer, pour chaque tâche, sur quelle machine et à quelle position elle doit être placée afin de minimiser $C_{\max}$.

Enjeux et applications

Ce problème est NP-difficile : aucune méthode exacte ne peut garantir l'optimalité pour des instances de grande taille. Il est très présent dans l'industrie :

— planification de production (ateliers flexibles),
— maintenance aéronautique,

Ces contextes exigent des solutions de bonne qualité en temps limité ce qui justifie l'utilisation d'heuristiques et de métaheuristiques.

3 Modélisation conceptuelle et mathématique

Les principales variables sont :

— p_{jk} : durée d'exécution de la tâche j sur la machine k,
— r_j : date de disponibilité,
— s_{ijk} : temps de setup entre les tâches i et j sur la machine k,
— $x_{jk} = 1$ si j est affectée à la machine k, 0 sinon.

Les contraintes principales sont les suivantes :

$$\sum_{k=1}^{m} x_{jk} = 1 \quad \forall j$$

Chaque machine exécute une seule tâche à la fois.

$$t_j \geqslant r_j \quad \forall j$$

La fonction objectif est :

$$\min C_{\max} = \min \max_{k \in M} \{C_k\}$$

où C_k désigne la date de fin de la dernière tâche exécutée sur la machine k.

4 Méthodes et algorithmes proposés

L'article combine des heuristiques de construction et des métaheuristiques d'optimisation afin d'obtenir des solutions performantes.

Heuristique BIBA (Best Insertion Based Approach)

La méthode BIBA construit une solution initiale faisable selon une logique gloutonne : à chaque étape, la tâche non encore planifiée qui augmente le moins le makespan est choisie et insérée à la position la plus avantageuse sur la machine correspondante. Elle fournit rapidement une solution initiale de bonne qualité.

Recherche locale VND (Variable Neighborhood Descent)

L'algorithme VND améliore la solution issue de BIBA en explorant successivement plusieurs voisinages :

— échange de deux tâches sur une même machine,
— insertion d'une tâche à un autre emplacement,
— déplacement inter-machine.

Chaque amélioration du $C_{\max}$ est acceptée jusqu'à convergence vers un optimum local.

Métaheuristiques globales

Pour échapper aux minima locaux, trois métaheuristiques sont étudiées :

— Late Acceptance Hill Climbing (LAHC) : accepte une solution moins bonne si elle n'est pas pire qu'une solution rencontrée il y a L itérations.
— Simulated Annealing (SA) : accepte temporairement des solutions plus mauvaises avec une probabilité liée à une température décroissante.

Schéma général :

```
S ← BIBA()
S ← VND(S)
S_best ← Metaheuristic(S)
```

5 Résultats et conclusion

Les expérimentations montrent que :

— les heuristiques seules (BIBA, VND) donnent des solutions correctes en peu de temps,
— les métaheuristiques améliorent significativement la qualité des solutions,
— la méthode LAHC offre le meilleur compromis entre performance et temps de calcul.

L'article conclut que la combinaison :

$$\text{BIBA} \rightarrow \text{VND} \rightarrow \text{LAHC}$$

constitue une approche efficace et modulaire pour le problème $R|r_j, s_{ijk}|C_{\max}$.

Cette stratégie guidera le développement du projet Java dont les prochaines étapes porteront sur la conception orientée objet et l'implémentation progressive des algorithmes.

II Conception logicielle

1 Diagramme UML

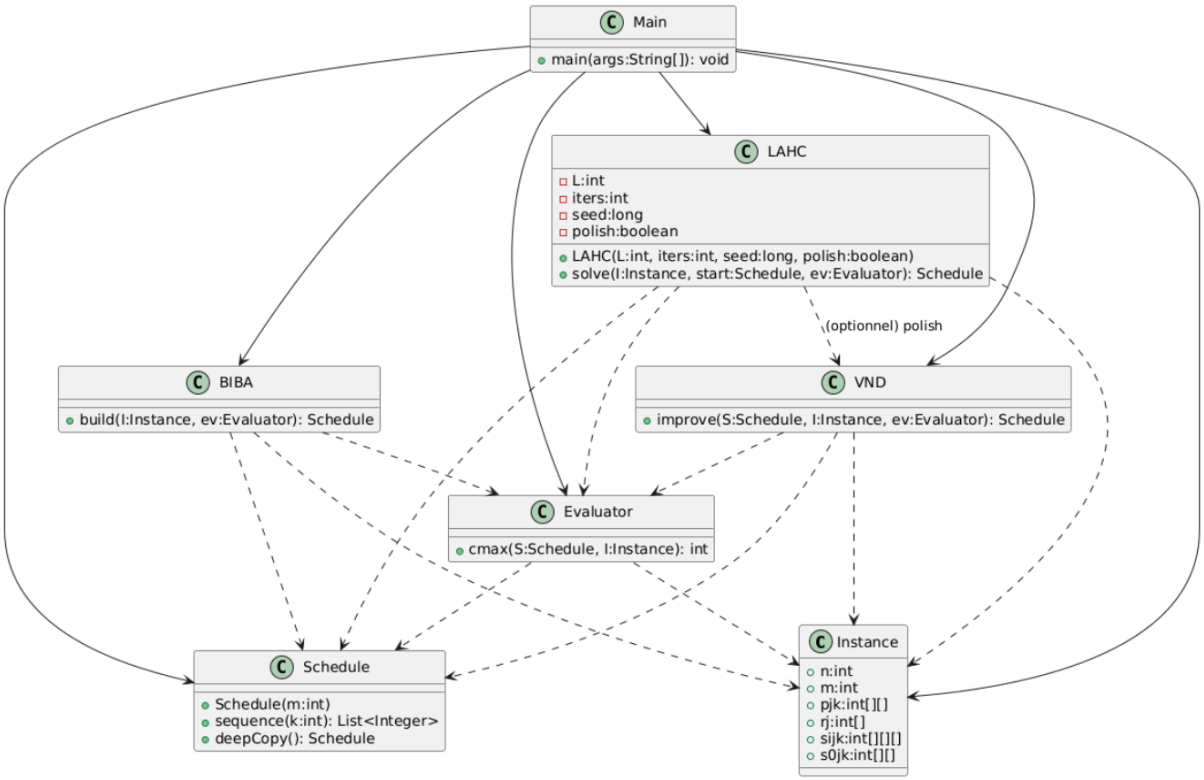


FIGURE II.1 – Diagramme de classes UML.

2 Description de l'architecture logicielle

L'architecture logicielle proposée traduit le modèle mathématique du problème $R \mid r_j, s_{ijk} \mid C_{\max}$ en un ensemble cohérent de classes Java orientées objet. Chaque classe joue un rôle bien défini et contribue à la modularité du système.

— **Instance** : contient toutes les données du problème, notamment les durées d'exécution p_{jk}, les dates de disponibilité r_j, les temps de préparation s_{ijk} ainsi que les temps de setup initiaux s^0_{jk}.

— **Schedule** : représente un ordonnancement sous la forme d'une liste de séquences de tâches par machine. Elle permet de créer, copier et manipuler des solutions candidates.

— **Evaluator** : calcule la valeur du makespan $C_{\max}$ pour un ordonnancement donné en tenant compte des contraintes liées aux dates de libération et aux temps de setup.
— **BIBA** : construit une solution initiale faisable en insérant les tâches une à une à la position minimisant l'augmentation du makespan. Cette heuristique fournit un point de départ de bonne qualité.
— **VND** : améliore la solution initiale à l'aide d'une recherche locale multi-voisinage (Variable Neighborhood Descent) et explore des mouvements tels que les échanges ou insertions de tâches.
— **LAHC** : implémente la métaheuristique *Late Acceptance Hill Climbing* qui permet d'échapper aux minima locaux en acceptant temporairement certaines dégradations selon un historique de taille L. Un polissage final peut être effectué via la méthode VND.
— **Main** : classe principale orchestrant l'ensemble du processus. Elle gère le chargement des données, l'appel successif des heuristiques (BIBA, VND, LAHC) et l'affichage des résultats.

Les flèches du diagramme de classes représentent les dépendances fonctionnelles entre les entités. Les classes d'algorithmes (*BIBA*, *VND*, *LAHC*) s'appuient sur *Evaluator* pour mesurer la qualité des solutions et utilisent les structures de données *Instance* et *Schedule* pour manipuler les ordonnancements.

Cette architecture modulaire favorise la réutilisation du code et la séparation claire des responsabilités ce qui garantit ainsi une meilleure lisibilité et maintenabilité du projet.

III Validation et tests

1 Objectif

Vérifier que l'implémentation respecte le modèle $R \mid r_j, s_{ijk} \mid C_{\max}$ et se comporte conformément à la méthode de l'article. La validation porte à la fois sur la faisabilité des solutions, la qualité du makespan et la stabilité du comportement des heuristiques.

2 Protocole de test

Les tests sont organisés en trois étapes.

1. Tests de faisabilité et de cohérence.
   — chaque tâche apparaît exactement une fois dans l'ordonnancement final ;
   — respect des dates de disponibilité r_j ;
   — non–chevauchement des tâches sur une machine.
2. Tests de performance sur des instances de tailles croissantes.
   — comparaison BIBA, VND et LAHC sur le même jeu de données ;
   — mesure du makespan $C_{\max}$ et du temps d'exécution.
3. Tests de robustesse.
   — réexécution avec différentes graines aléatoires pour LAHC ;
   — variantes avec ou sans temps de setup.

3 Jeux d'essai

Trois benchmarks synthétiques sont utilisés.

I1, petite instance sans setup

$$n = 6,\ m = 2, \quad p = \begin{bmatrix} 6 & 7 \\ 5 & 5 \\ 7 & 6 \\ 4 & 8 \\ 9 & 3 \\ 6 & 6 \end{bmatrix}, \quad r = \begin{bmatrix} 0 & 2 & 0 & 3 & 1 & 0 \end{bmatrix}$$

Aucun setup : $s_{ijk} = 0$ et $s^0_{jk} = 0$.

I2, instance avec setup unitaire

$$n = 8,\ m = 3, \quad p = \begin{bmatrix} 3 & 9 & 6 \\ 4 & 4 & 7 \\ 5 & 6 & 5 \\ 7 & 8 & 4 \\ 2 & 2 & 9 \\ 9 & 3 & 6 \\ 4 & 5 & 5 \\ 6 & 4 & 7 \end{bmatrix}, \quad r = \begin{bmatrix} 0 & 0 & 1 & 2 & 0 & 3 & 1 & 2 \end{bmatrix}$$

Setup séquentiel : $s_{ijk} = 1$ si $i \neq j$, 0 sinon. Pas de setup de démarrage.

I3, instance plus grande et déséquilibrée

$n = 12,\ m = 3, \quad p_{jk} \in [2, 12]$ avec disparités par machine, $\quad r_j \in [0, 5], \quad s_{ijk} = 1_{\{i \neq j\}}$

Les valeurs détaillées sont générées de manière déterministe à partir d'une graine fixée.

4 Mesures relevées

Pour chaque instance et chaque méthode, les indicateurs suivants sont collectés.
- makespan final $C_{\max}$;
- temps d'exécution en millisecondes ;
- séquences par machine (pour vérification visuelle).

5 Tableaux de (Executer code)

Instance	n	m	BIBA	VND	LAHC	Commentaire
I1	6	2	16	16	16	Instance simple, pas d'amélioration
I2	8	3	13	13	13	Setups unitaires, plan déjà équilibré
I3	12	3	25	22	22	Amélioration nette par VND/LAHC

Temps moyens (ms) mesurés : I1 : $t_{\text{BIBA}} = 1.20$, $t_{\text{VND}} = 0.21$, $t_{\text{LAHC}} = 9.04$; I2 : 3.31, 0.28, 9.39 ; I3 : 0.56, 0.45, 11.13.

Séquences finales (solution LAHC retenue) :
I1 : $M_0 = [5, 0, 3]$, $M_1 = [2, 4, 1]$.
I2 : $M_0 = [0, 6, 4]$, $M_1 = [1, 7, 5]$, $M_2 = [2, 3]$.
I3 : $M_0 = [5, 8, 6]$, $M_1 = [3, 7, 4, 1, 2]$, $M_2 = [0, 9, 10, 11]$.

6 Critères d'acceptation

- faisabilité satisfaite sur toutes les instances (chaque tâche apparaît exactement une fois, aucune préemption, respect des r_j via l'évaluateur) ;
- en tendance, VND ne dégrade pas BIBA et LAHC est au plus égal à VND : confirmé sur I1 et I2 (égalité) et sur I3 (amélioration de BIBA à VND/LAHC) ;
- stabilité de LAHC sur plusieurs graines : I1, I2 et I3 donnent $\text{LAHC}_{\min} = \text{LAHC}_{\max}$ (16, 13, 22) donc variance nulle sur les 3 graines testées.

7 Analyse

Le comportement observé est conforme aux attentes du modèle $R \mid r_j, s_{ijk} \mid C_{\max}$ et aux heuristiques mises en oeuvre.

- **I1 (simple, sans setup).**La construction gloutonne BIBA atteint déjà une solution quasi optimale que VND/LAHC ne peuvent pas améliorer avec des voisinages locaux.
- **I2 (setups unitaires).** Bien que les setups pénalisent les changements de série, la solution BIBA obtient un ordonnancement déjà équilibré par machine et compatible avec les r_j ; aucun mouvement local n'abaisse le makespan d'où l'égalité BIBA=VND=LAHC.
- **I3 (plus grande, hétérogène).** Les vitesses différentes entre machines et la dispersion des r_j créent des déséquilibres ; VND réorganise utilement les séquences (échanges/insertions) et réduit $C_{\max}$ de 25 à 22. LAHC confirme cette solution (min=avg=max=22), montrant une recherche globale stable.

En synthèse, BIBA fournit un point de départ rapide et solide, VND améliore lorsque l'instance présente un vrai espace d'amélioration et LAHC consolide la meilleure solution atteinte avec une variabilité très faible sur les graines testées.